CIT CAMPUS PLATFORM

API Specification & Backend Domain Contracts

Technical contract proposal · Version 0.1 · Not frozen

API: /api/v1
Depends on: Source Audit, Data Dictionary and ERD/Database Design v0.1
Backend proposal: FastAPI + Pydantic + SQLAlchemy + PostgreSQL

Defines endpoints, request/response contracts, permissions, state machines, transactions, AI boundaries, validation and testing for the CIT competition platform.

Authority and status

The official starter-pack JSON, example export and validator.py define the competition input/output contract. This document defines proposed application API and backend behaviour. It is not frozen; do not generate production code until reviewed.

1. Goals and non-goals

• Stable communication contracts between React frontend and FastAPI backend.

• Business rules live in backend domain services, not UI or AI prompts.

• Every state-changing action is authenticated, authorized, validated and audited.

• Support imports, timetable, exams, bookings, disruption repair, exact exports and Campus Navigator.

• AI creates drafts/proposals; backend and authorised users remain authoritative.

• This document does not decide unknown reviewer routing, add competition roles, promise recurring/multi-room bookings, or prescribe final UI design.

2. Global API conventions

• Base path `/api/v1`; breaking changes require versioning.

• Actor identity and role/scope come from authenticated credentials, never from client-supplied user_id/role/is_admin.

• Use Authorization bearer token if selected, JSON content type, Idempotency-Key for submissions/jobs/decisions/publish, and If-Match or expected_version for concurrency.

• Success envelope: data + meta.request_id/api_version. Lists add page/page_size/total. Errors return stable code, safe message, details and retryable.

• Default page size 25, maximum 100 (proposed); stable sort required.

• API timestamps use RFC3339; teaching slot uses day/start_period/duration; booking end period inclusive; exam uses day/session AM/PM.

• Expose internal UUID and external competition ID separately; never construct IDs from names or array positions.

3. Roles and permissions

Capability	Admin	Dept Coord	Faculty	Club Rep
View institute timetable	Yes	Scoped/policy	No by default	No
Own timetable	Yes	Yes	Yes	No
Import/generate timetable	Yes	No	No	No
Manual timetable edit	Yes	Scoped proposal	No	No
Submit unavailability	Yes	No	Own only	No
Department session change	Yes	Own dept	No	No
Venue request	Yes	Department events	Faculty category	Club category
Booking decision	Authorised reviewer only	Only if policy grants	No self-approval	No self-approval
Approve disruption repair	Yes	No	No	No
Publish timetable/map	Authorised admin	No by default	No	No

Official competition roles are admin, department_coordinator, faculty and club. Student read access can be a product access policy, not an added competition role. Scope checks are mandatory. The source does not fully specify reviewer routing; treat it as open policy.

4. State machines

Timetable: DRAFT → GENERATING → GENERATED → VALIDATING → VALIDATED → APPROVED → PUBLISHED; failures/timeouts recorded. A schedule with hard violations cannot be publishable.

Booking: keep request lifecycle separate from decision. Request: draft/submitted/under_review/decided/cancelled/withdrawn. Export decision status must be approved/rejected/alternative_offered. Alternative acceptance/decline is an application extension whose export semantics remain open.

Approval task: pending → assigned → in_review → approved/rejected/changes_requested/cancelled. Disruption: draft → submitted → validating → validated → pending_admin_approval → approved → applying → applied (or failed/rejected). Map: draft → processing → review_required → validated → approved → published → superseded.

5. Endpoint catalogue

Domain	Endpoints (relative to /api/v1)
Auth	GET /auth/me; POST /auth/session/refresh; POST /auth/logout
Dataset	GET /datasets; POST /datasets/import; GET /datasets/{id}; GET /datasets/{id}/configuration; POST /datasets/{id}/activate; GET /datasets/{id}/import-report
Academic	GET /rooms[/{id}]; GET /faculty[/{id}]; GET /student-groups; GET /courses; GET /sessions[/{id}]
Timetable	POST /timetables/generation-jobs; GET /timetables; GET /timetables/{id}; GET /timetables/{id}/assignments; GET .../views/groups/{id}; .../faculty/{id}; .../rooms/{id}; POST .../validate|approve|pub
Jobs	GET /jobs/{id}; POST /jobs/{id}/cancel; GET /solver-runs/{id}; GET /solver-runs/{id}/metrics
Bookings	POST /bookings/parse; POST /bookings; GET /bookings[/{id}]; POST /bookings/{id}/submit|cancel|decisions|alternative-response
Approvals	GET /approvals/inbox; GET /approvals/{id}; POST /approvals/{id}/actions
Exams	POST /exam-schedules/generation-jobs; GET /exam-schedules[/{id}]; POST /exam-schedules/{id}/validate|approve|publish
Disruptions	POST /disruptions/import; GET /disruptions[/{id}]; POST /disruptions/{id}/validate; POST /change-proposals; GET /change-proposals/{id}[/diff]; POST .../validate|approve|reject|apply
Navigator	GET /campus-map/versions|current; POST /admin/campus-map/assets; POST .../process; GET .../review-items; POST .../decision|validate|approve|publish|rollback; GET /places/search|{id}; PO
Assistant	POST /assistant/conversations; GET /assistant/conversations/{id}; POST .../{id}/messages; GET /assistant/tool-executions/{id}
Exports	POST /exports/schedule; POST /exports/schedule-after; GET /exports/{id}; POST /exports/{id}/validate; GET /validation-runs/{id}

6. Detailed endpoint contracts

Dataset import — `POST /datasets/import` (Admin)

```
Request: multipart file=cit_public.json, activate=false, dry_run=false
Response:
{
  "data": {
    "import_job_id": "uuid",
    "dataset_version_id": "uuid",
    "external_dataset_id": "cit-public-v1",
    "status": "completed",
    "counts": {"rooms":27,"faculty":40,"student_groups":14,
      "courses":62,"sessions":189,"booking_requests":25,
      "cancellations":1,"exams":41},
    "warnings": []
  }
}
```

Validate root keys/types and references; retain original checksum; import atomically; do not overwrite immutable data; dry-run returns a report without activating the dataset.

Timetable generation — `POST /timetables/generation-jobs`

```
{
  "dataset_version_id": "uuid",
  "base_timetable_version_id": null,
  "objective_profile": "balanced",
  "time_limit_seconds": 60,
  "random_seed": 17,
  "include_explanations": true
}
Response: { "data": { "job_id":"uuid", "solver_run_id":"uuid",
  "status":"queued", "status_url":"/api/v1/jobs/uuid" } }
```

Job statuses: queued/running/succeeded/failed/cancelled/timed_out. Return assignments, unassigned IDs, hard count, S1–S6 breakdown, objective/bound, runtime and validator report. Solver feasibility alone does not mean official validator acceptance.

Timetable read views

Provide group, faculty and room views. Each item includes source session ID, course, day, start/end period, clock times, room and faculty. Week 7 view incorporates only validated session overrides.

Manual assignment

```
{
  "expected_timetable_version": 3,
  "assignments": [{
    "session_external_id":"MA101-AB-L1",
    "day":"MON","start_period":2,"room_external_id":"LH-1"
  }],
  "reason":"Approved manual adjustment"
}
```

Validate all hard constraints; use optimistic concurrency; no partial writes by default; record before/after and run official validator. Department Coordinator proposal-only is the safe default until policy is confirmed.

Natural-language booking parse — `POST /bookings/parse`

```
{
  "text":"Need a room for 45 people on Tuesday 6 to 8 pm, projector please.",
  "locale":"en-IN",
  "reference_datetime":"2027-01-26T10:00:00+05:30",
  "dataset_version_id":"uuid"
}
Response draft: attendance=45, duration_periods=2,
window=TUE P10-P11, needs=[projector],
room_types=[classroom,lecture_hall,seminar_hall],
mandatory_for_groups=[], missing_fields=[], assumptions=[],
follow_up=null, requires_user_confirmation=true
```

Parsing creates no booking and reserves no room. User edits/confirms draft. Resolve relative dates using explicit reference datetime and campus timezone. Hindi `kal` follows sample convention as tomorrow, but show resolved weekday. Vague requests require a follow-up or visible assumption. Accessibility/name conflicts must be surfaced; never silently substitute.

Booking creation and decision

```
Create draft:
{
  "title":"Coding contest","attendance":60,"duration_periods":3,
  "windows":[{"day":"THU","from_period":6,"to_period":9}],
  "room_types":["lab_cs"],"needs":["projector"],
  "mandatory_for_groups":[],
  "requested_room_external_id":null,
  "raw_text":"..."
}
Decision:
{
  "status":"alternative_offered",
  "room_external_id":"AUD","day":"THU",
  "start_period":10,"end_period":12,
  "reason":"Requested hall unavailable; auditorium meets requirements."
}
```

Backend derives role/category from actor and dataset caps. Submission does not reserve a room. Decision checks reviewer scope, no-self-approval, cancellation, stale version, effective priority, room event permission/type/capacity/features, time window, unavailability, timetable, existing bookings and mandatory groups. Recheck inside transaction.

Exam generation

Generate against exam calendar. Every exam exactly once; valid day/session; rooms allow exams; half capacity is floor(room.capacity/2); multiple rooms allowed; no room/group overlap per day/session; same-day group soft penalty 5 per additional exam. Preserve duration_hours; official validator does not explicitly compare it to AM/PM window.

Disruption repair

Import exact supported changes: faculty_unavailable, room_unavailable, add_faculty, add_session, remove_session. Preserve change order. Proposal binds baseline timetable and target dataset. Validate no hard violations, fixed slots, added/removed sessions; compare before/after; minimize moved sessions then soft score as a proposed lexicographic objective. Apply only after Admin approval and revalidate if baseline changed.

Campus map and navigation

Upload assets, process to draft map, review items, validate geometry and graph, approve and publish immutable version. Validate geometry, floor coordinate system, room links, entrances, edge endpoints, disconnected nodes, accessibility and route reachability. Outdoor WGS84 and indoor local coordinates stay separate. Indoor origin requires manual selection or a supported QR/positioning source; GPS alone cannot identify exact indoor location.

AI assistant

Gemini may read permitted context, parse requests, propose changes and explain validation. Backend validates tool schemas and permissions, runs domain validation, asks for confirmation, and uses normal approval flow. Never expose unrestricted SQL, direct DB mutation, approval bypass or publish tools.

Export

Exact competition export fields: top-level team/dataset_id/sessions/fest_week/exams; session_id/day/start_period/room_id; booking statuses exactly approved/rejected/alternative_offered; inclusive end_period; overrides only as permitted; exam_id/day/session/rooms. Run pinned official validator on exact serialized output and store report, version/hash and checksum. No manual edits.

7. Domain service boundaries

Service	Responsibility
DatasetImportService	Parse, validate and import source atomically
AuthorizationService	Roles, scopes, permissions, no-self-approval
SchedulingConstraintService	Hard/soft validation and conflict checks
TimetableGenerationService	Solver job and timetable version
TimetableRepairService	Derived dataset and minimal repair proposal
BookingParserService	Gemini parse into editable draft
BookingDecisionService	Priority, allocation, transactional decision
ApprovalWorkflowService	Tasks, routing, events and audit
ExamSchedulingService	Exam slots and multi-room allocation
ExportService / OfficialValidatorService	Exact JSON and official validator reports
CampusMap services	Ingestion, validation, review, publish
NavigationRoutingService	A*/Dijkstra over published graph

Service	Responsibility
AssistantOrchestrator	Constrained tool orchestration; no direct DB writes

8. Transactions and concurrency

• Booking: lock request and contested room/time; re-check identity, cancellation, priority, availability and conflicts; write decision/allocation/audit atomically; notify after commit.

• Timetable publish: compare active version; require zero hard violations and authorised approval; atomically switch published pointer and audit.

• Disruption apply: require Admin approval; ensure baseline current or revalidate; atomically activate derived dataset/timetable; retain prior versions.

• Idempotency: persist key, actor and endpoint for import, generation, booking, decisions, apply, publish and export. Identical replay returns original result; changed payload with same key returns 409.

9. Error catalogue

HTTP	Code	Meaning
400	INVALID_REQUEST	Malformed payload or basic value
401	UNAUTHENTICATED	Missing/invalid credentials
403	FORBIDDEN	Role/scope denied
404	RESOURCE_NOT_FOUND	Missing/not visible
409	VERSION_CONFLICT	Stale resource version
409	RESOURCE_CONFLICT	Room/time already occupied
409	IDEMPOTENCY_KEY_REUSED	Key reused with different payload
422	DOMAIN_VALIDATION_FAILED	Domain rules failed
422	SCHEDULE_HARD_CONFLICT	Hard timetable rule failed
422	BOOKING_NOT_ELIGIBLE	Cancelled/invalid/priority constraints
422	IMPORT_REFERENCE_ERROR	Unresolved source reference
422	MAP_VALIDATION_FAILED	Geometry/graph validation failed
429	RATE_LIMITED	Throttled
500	INTERNAL_ERROR	Unexpected safe error
503	DEPENDENCY_UNAVAILABLE	Solver/AI/storage unavailable

10. Background jobs

Use jobs for timetable/exam/repair solvers, official validation, large imports/exports and map processing. A simple worker/database queue is enough initially; add Redis/Celery/RQ only if justified. Persist queued/started/completed, status, progress, result resource, safe error, retries and worker version. Notify after durable commit; use an outbox if delivery reliability requires it.

11. Test and acceptance strategy

• Contract tests for schemas, IDs, pagination and permissions.

• Import counts, references, nulls, checksums, idempotency and rollback.

• H1–H12/H14, basket occupancy, intervals, lunch, labs, unavailability, fixed slots and S1–S6.

• Booking role caps, timestamp ties, cancellation B3, windows, alternatives, features, capacity, mandatory groups, priority inversion, concurrency and no-self-approval.

• Exam exactly-once, half-capacity floor, multi-room capacity and group/room conflict.

• All disruption types, order, derived version, stale baseline and Admin-only apply.

• Navigator coordinate systems, graph endpoints, reachability, accessibility, map immutability and rollback.

• AI schema/permission validation; no direct writes or publish.

• End-to-end import → timetable → export → official validator → disruption → repair → schedule_after export → official validator with baseline.

12. Implementation order

• Confirm stack and policy decisions.

• Freeze schema and API contracts.

• Set up repository, environment, PostgreSQL, SQLAlchemy and Alembic.

• Implement dataset import and read APIs.

• Wrap official validator and persist reports.

• Implement solver, timetable versions and export adapter.

• Prove schedule.json validator compatibility.

• Implement booking/approval, exams, disruptions/repair.

• Build frontend vertical slices against stable APIs.

• Implement Navigator and Gemini.

• Run end-to-end/security/performance tests and deploy.

13. Open decisions

ID	Question	Current status
API-01	Team Python or TypeScript capability?	FastAPI conditional
API-02	Exact booking reviewer routing?	Organiser policy needed
API-03	Alternative accept/decline and export semantics?	Open product extension
API-04	Admin approval for timetable publish?	Proposed gate
API-05	Coordinator direct edit or proposal only?	Proposal-only default
API-06	Institute-wide timetable visibility?	Policy needed
API-07	Student read access mechanism?	Product policy
API-08	Recurring bookings?	Defer
API-09	Multi-room venue bookings?	Not current export contract
API-10	Indoor positioning?	Manual origin/QR optional
API-11	Worker queue?	Start simple
API-12	Notification channels?	In-app first

14. Milestone acceptance checklist

■ Endpoint paths/methods reviewed.

■ Request/response schemas reviewed.

■ Official roles and scopes approved.

■ Reviewer routing clarified.

■ Alternative semantics clarified.

■ Timetable/exam/disruption workflows approved.

■ Errors and concurrency rules approved.

■ AI boundaries approved.

■ Export/validator integration approved.

■ Team stack confirmed.

■ OpenAPI generated from approved Pydantic schemas.

■ Contract tests pass.

Next: review/freeze architecture decisions, then prepare implementation-ready backend foundation and migration plan. No application code until decisions affecting schema and permissions are resolved.